포팅 메뉴얼(C108)

1. 사용한 JVM, 웹서버, WAS 제품 종류와 설정값, 버전(IDE 버전 포함)

- **JVM**: eclipse-temurin:17-jdk
- **웹서버**: Nginx 1.24.0 (Reverse Proxy /api 경로 → Spring Boot 컨테이너)
- **WAS**: Spring Boot 내장 Tomcat 3.2.x
- **IDE**: IntelliJ IDEA Ultimate 2025.1.3
- **설정**:
  - JAVA_OPTS=-Xms512m -Xmx1024m -Dfile.encoding=UTF-8
  - Nginx HTTPS 적용 (Let's Encrypt 인증서 /etc/letsencrypt/live 경로)
  - Jenkins는 Docker 컨테이너로 실행 (jenkins/jenkins:lts)

2. 빌드시 사용되는 환경 변수

- FrontEnd 환경변수

```
VITE_YORKIE_API_ADDR='https://api.yorkie.dev'
VITE_YORKIE_API_KEY='Wq6BpKh4m9YUV2tWmEqC9t'
VITE_API_URL="https://i13c108.p.ssafy.io"
VITE_AUTH_REDIRECT_URI="http://i13c108.p.ssafy.io:8082/oauth/callback"
VITE_GITHUB_CLIENT_ID="Ov23liutbja141i00hCI"
VITE_GITHUB_IMPORT_URL="https://github.com/apps/ottereviews/installations/new"
```

- Backend(spring boot) 환경변수

```
MYSQL_DATABASE=ottereview
MYSQL_USER=ssafy
MYSQL_PASSWORD=F8hHSWEawgBsnU0uDJifUh0zuorr8VGfkyx5gWdKOpLa7ACbv9
MYSQL_ROOT_PASSWORD=7KFhWC1TpnpUAFRrr2l9aYXe6erjZ95W1vb06pal8QJXVuFaly

SPRING_DATASOURCE_URL=jdbc:mysql://db-mysql:3306/ottereview?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
SPRING_DATASOURCE_USERNAME=ssafy
SPRING_DATASOURCE_PASSWORD=F8hHSWEawgBsnU0uDJifUh0zuorr8VGfkyx5gWdKOpLa7ACbv9

GITHUB_APP_CLIENT_ID=Iv23li1CkOpJF4kPZhGv
GITHUB_APP_CLIENT_SECRET=ea5bc2b0a81817bbd227728f64718f47081030c1
GITHUB_APP_OAUTH_REDIRECT_URI =https://i13c108.p.ssafy.io/api/github-app/installation/callback
GITHUB_APP_ID=1734805
GITHUB_APP_PRIVATE_KEY_PATH=file:/app/config/github-app-private-key.der
GITHUB_APP_AUTHENTICATION_JWT_ACCESS_EXPMIN=60000

GITHUB_OAUTH_CLIENT_ID=Ov23liutbja141i00hCI
GITHUB_OAUTH_CLIENT_SECRET=e9627a8480c6c2a81ca4e5b9225c9987151a6cc3
GITHUB_OAUTH_REDIRECT_URI=https://i13c108.p.ssafy.io/oauth/github/callback

APP_JWT_SECRET=9kMzDX83GI5+vuI4jbpOW5Xdatg91TCS2rmcaAYW19M=
APP_JWT_ACCESS_EXPMIN=30
APP_JWT_REFRESH_EXPMIN=10080

S3_ACCESS_KEY=AKIA6H6YGWTKD6DH6DFW
S3_SECRET_KEY=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
S3_BUCKET_NAME=ottereview
S3_REGION=ap-northeast-2
```

```
OPENVIDU_SECRET=ottereview
OPENVIDU_URL=https://i13c108.p.ssafy.io:9001

EMAIL_USERNAME=ottereview@gmail.com
EMAIL_PASSWORD=nnmfqcswztzeksly

AI_SERVER_BASE_URL=http://ottereview-ai:8000
```

- Backend(ai 서버) 환경변수

```
OPENAI_API_KEY=S13P11C108-20da8a70-eb37-49bd-ad88-1f1cadb3a73b
PINECONE_API_KEY=pcsk_2Q2vt3_2AD89au7UXGVtTLvwjQyVDHkFsd4GGJo5U3WHjP2DK5dqRcdJPE3Y3HPNBtG13a
REDIS_HOST=ottereview-redis
REDIS_PORT=6379
REDIS_DB=0
```

3. 배포 시 특이사항 기재

- **빌드 산출물**: `build/libs/app.jar` → Docker 이미지 빌드 시 COPY
- **배포 방식**: Jenkins → Docker Build → Docker compose Build & Run
- **주의 사항**:
  1. `.env` 파일은 **Git에 포함 금지** (Jenkins에서만 관리)
  2. MySQL/Redis , AI 서버 , Spring 서버, Openvidu 컨테이너와 동일 네트워크 사용 (Docker Compose `networks`)
  3. `docker compose down && docker compose up -d` 로 무중단 배포
  4. HTTPS 인증서 90일마다 갱신 필요 (`certbot renew`)
  5. `.private_key.per` 파일은 github app 인증을 위한 키로 Git에 포함 금지(Jenkins Credentials로 관리)

- 배포 특징 정리
  1. Jenkins Item을 3개로 AI , Be(spring), FE 이렇게 따로 두고 CI/CD 각각 되게 설정하였음
  2. nginx를 두개 놓고 하나는 443(https)를 Listen하고 다른 하나는 80(http)를 Listen하게 해서 각각 리버스 프록시를 하게 해놨음
- 포트 사용 현황
  - spring boot : 8080
  - jenkins : 8081
  - Frontend : 8082
  - AI 서버 : 8000
  - Mysql : 3307
  - redis : 6380
- Jenkins 빌드 명령어 정리
  - Front-end (jenkins pipeline items 사용)

```
properties([
  pipelineTriggers([
    gitlab(
      triggerOnPush: true,
      triggerOnMergeRequest: true,
      branchFilterType: 'NameBasedFilter',
      includeBranches: 'master'
    )
  ])
])

pipeline {
    agent any

    tools {
        nodejs "node-18"
    }

    environment {
```

```
        IMAGE_NAME     = "frontend-app"
        CONTAINER_NAME = "frontend-container"
        REMOTE_HOST    = "i13c108.p.ssafy.io"
        REMOTE_USER    = "ubuntu"
        SSH_KEY_ID     = "ec2-ssh-key"  // Jenkins에 등록된 SSH credential
s ID
        BUILD_DIR      = "build"        // React: build, Vue/Vite: dist
    }

    options {
        // 기본 체크아웃 비활성화 (명시적 git 스텝만 사용)
        skipDefaultCheckout(true)
    }

    stages {

        stage('Checkout') {
            steps {
                git branch: 'master',
                    url: 'https://lab.ssafy.com/s13-webmobile1-sub1/S13P11C10
8.git',
                    credentialsId: 'ottereview123'
            }
        }

        stage('Build Frontend') {
            steps {
                dir('ottereview-fe') {
                    sh 'rm -f .env || true'        // 기존 파일 삭제(없어도 무시)
                    sh 'chmod u+w . || true'
                    withCredentials([file(credentialsId: 'FRONTENV', variable: 'F
RONT_ENV_FILE')]) {
                        // 프로젝트 루트에 .env 복사
                        sh 'cp "$FRONT_ENV_FILE" .env'
                    }
                    sh '''
                        npm install
                        npm run build
```

```
            '''
        }
    }
}

stage('Docker Build') {
    steps {
        dir('ottereview-fe') {
            sh """
                docker build -t ${IMAGE_NAME} .
            """
        }
    }
}

stage('Transfer Image to EC2') {
    steps {
        withCredentials([file(credentialsId: 'FRONTENV', variable: 'FR
ONT_ENV_FILE')]) {
            sh "docker save -o ${IMAGE_NAME}.tar ${IMAGE_NAME}"
            sshagent(credentials: [SSH_KEY_ID]) {
                sh """
                  scp -o StrictHostKeyChecking=no ${IMAGE_NAME}.tar
\
                      ${REMOTE_USER}@${REMOTE_HOST}:~/
                  scp -o StrictHostKeyChecking=no "$FRONT_ENV_FILE"
\
                      ${REMOTE_USER}@${REMOTE_HOST}:~/.env
                """
            }
        }
    }
}

stage('Deploy to EC2') {
    steps {
        sshagent(credentials: [SSH_KEY_ID]) {
            sh """
```

```
                    ssh -o StrictHostKeyChecking=no ${REMOTE_USER}@
${REMOTE_HOST} << 'EOF'
                        docker stop ${CONTAINER_NAME} || true
                        docker rm ${CONTAINER_NAME} || true
                        docker load -i ${IMAGE_NAME}.tar
                        docker run -d \\
                          --name ${CONTAINER_NAME} \\
                          -v /home/ubuntu/.env:/app/.env:ro \\
                          -p 8082:80 \\
                          ${IMAGE_NAME}
EOF
                """
            }
        }
      }
    }
}
```

- Backend(freestyle 사용)

```
# workspace 내에서 작업
mkdir -p /var/jenkins_home/workspace/ottereview/secrets

cat "$DOTENV_FILE" > /var/jenkins_home/workspace/ottereview/secre
ts/.env
cat "$GITHUB_PRIVATE_KEY_FILE" > /var/jenkins_home/workspace/ott
ereview/secrets/github-app-private-key.der

cp /var/jenkins_home/workspace/ottereview/secrets/.env /var/jenkins_
home/workspace/ottereview/ottereview-be/.env
cp /var/jenkins_home/workspace/ottereview/secrets/github-app-privat
e-key.der /var/jenkins_home/workspace/ottereview/ottereview-be/gith
ub-app-private-key.der

cd ottereview-be
chmod +x gradlew
./gradlew clean build
```

```
docker build -t kyoungil/ottereview:latest -f Dockerfile .
docker compose down
docker compose up -d
docker compose exec springboot-app sh -c 'printenv | grep GITHUB_OAUTH_REDIRECT_URI'
```

- AI 서버(freestyle 사용)

```
# 1. Jenkins 워크스페이스의 ottereview-ai 디렉토리로 이동
# Git 리포지토리가 이 디렉토리를 생성하므로, 이 위치가 프로젝트의 루트가 됩니다.
cd /var/jenkins_home/workspace/ottereview_ai/ottereview-ai

# 2. .env 파일을 바로 현재 디렉토리에 생성
# Jenkins 환경 변수 AIENV의 내용을 .env 파일로 직접 리디렉션합니다.
# 이렇게 하면 cp 명령 없이도 .env 파일이 정확한 위치에 만들어집니다.
cat "$AIENV" > .env

# 3. docker compose 실행
# 기존 컨테이너를 내리고, 새로 빌드 및 실행합니다.
docker compose down
docker compose up --build -d
```